Renee Banner & Alex Guerra

Lab: Tracing a Response from w.Write() to the Wire

Experiment 1: follow one small response from `Write` to the socket

Goal: see, line by line, that a small response leaves your program in **one** `write()` call, and that it leaves **after** your handler has returned. This experiment is fully worked: every log line is explained. The later experiments ask you to do more of the reading yourself.

The handler under test is `hello`. It makes one call: `fmt.Fprintln(w, "hello from the lab server")`. That text plus a newline is 26 bytes.

Step 1: predict (write your answers on paper)

1. How many `SOCKET conn.Write` lines will appear for this one request: 0, 1, 2 or 26?
    a. 1
2. Will the `SOCKET conn.Write` line appear **before** or **after** the line `handler returns`?
    a. after
3. The body is 26 bytes. Will the `SOCKET` line say 26, or something else? If something else, what is the rest?

    a. More than 26 bytes due to extra information

Use the model section to reason it out. Do not look ahead.

Step 5: explain it in your own words

Write two or three sentences for each:

1. Why does the response say `Content-Length: 26` and not `Transfer-Encoding: chunked`? (Hint: what did the chunkWriter know at the moment it built the header?)
    a. It says Content-Length: 26 because the handler is aware of the size of the data when it is returned. Transfer-Encoding: chunked is used when the handler returns before having a complete idea of the size of the data.
2. Which log line is header moment 1, and which is header moment 3? Why is moment 2 not in the log?
    a. Moment 2 happens inside Go, between Buffer A and Buffer B. The instruments sit only above A and below B, so nothing is logged there. The 117 extra bytes are the evidence.
3. Your handler finished before the client got any bytes. Which box held your 26 bytes at the moment the handler returned? What does this tell you about the phrase "I already wrote the response"?

    a. means the bytes were copied into a Go-internal buffer. The client had received nothing.

Experiment 2: watch the two buffers fill, in slow motion

Goal: see Buffer A and Buffer B fill up and overflow, one `Write` at a time, and see the moment the client first receives data.

Renee Banner & Alex Guerra

The handler under test is `drip`. It makes 8 `Write` calls of 1,000 bytes each, with a 300 millisecond pause after each one. Because of the pauses, you can read the log as it appears. The whole request takes about 2.4 seconds.

Step 1: predict

The sizes you need: Buffer A holds 2,048 bytes, Buffer B holds 4,096 bytes. Each `Write` adds 1,000 bytes to A. The header will add about 120 bytes and each chunk adds a few bytes of framing, so you can ignore those when estimating.

Write your answers on paper:

1. After `Write #2` (2,000 bytes written in total), has anything reached the socket? Why or why not?
    a. No, 2,000 bytes is under A's 2,048 limit.
2. Which `Write` makes A overflow for the first time? Where do its bytes go?
    a. The second Write maids A overflow. A empties into the chunkWriter, then into B, and the header is built
3. Which `Write` makes B overflow for the first time? What will that first `SOCKET` line say, how many bytes, and what will they start with?
    a. The 3rd write will make B overflow. The first socketline will say: 143 bytes starts "HTTP/1.1 200 OK\r\nDate: Mon, 19 Oct"
4. This handler runs for about 2.4 seconds. After how many seconds will the client first receive anything? Pick one: right away, about 1.2 seconds, about 2.4 seconds.
    a. about 1.2 seconds
5. Will the response use `Content-Length` or `Transfer-Encoding: chunked`?

    a. Transfer-Encoding: chunked

Step 7: explain it in your own words

1. Why did the 1,000-byte version use `chunked`, while the 100-byte version used `Content-Length`? Use the words "Buffer A" and "the chunkWriter" in your answer.
    a. With 1,000-byte writes, Buffer A overflowed during the handler, so the chunkWriter had to build the header before it knew the total length and chose chunked. With 100-byte writes, A never filled. The handler returned first, the chunkWriter saw all 800 bytes, and it wrote Content-Length.
2. The two runs have the same handler code, the same pauses and the same total time. Why does the client see the first byte at different times?
    a. The code is the same, but the buffers behave differently. In the 1,000-byte run the data overflowed A and then B, which forced an early write() at Write #5. In the 100-byte run nothing overflowed, so nothing left until the handler returned.
3. A web page that takes 3 seconds to build, written in small `Write` calls without `Flush`, shows the user a blank screen for 3 seconds. Which box is holding the data during that time, and what two things could the developer do about it? (Experiment 4 will test one of them.)

    a. Buffer A (and later B) holds the data. The developer could call Flush() after each piece to send early, or build the page faster by moving the slow work (queries) before the first

Write and then writing everything at once. A third option is to send a fast shell and load the slow parts separately.

Experiment 3: many small writes against one big write

Goal: discover that the number of `w.Write` calls your code makes has almost nothing to do with the number of `write()` calls that reach the kernel. The buffers decide that, not your code.

Two handlers:

- `pieces` makes `n` separate `Write` calls of 10 bytes each. `pieces?n=100` writes 1,000 bytes in total.
- `big` makes **one** `Write` call of `kb` kilobytes. `big?kb=64` is one call of 65,536 bytes.

Step 1: predict

Copy this table onto paper and fill in the last two columns **before** running anything. "SOCKET lines" means how many `SOCKET conn.Write` lines will appear for that request. Remember the sizes: A holds 2,048 bytes, B holds 4,096 bytes.

Request	Write calls	Body bytes	Your guess: SOCKET lines	Your guess: Content-Length or chunked
`/pieces?n=100`	100	1,000	1	content-length
`/pieces?n=300`	300	3,000	2	chunked
`/pieces?n=1000`	1,000	10,000	3	chunked
`/big?kb=1`	1	1,024	1	content-length
`/big?kb=64`	1	65,536	19	chunked

Two things to think about while you guess. First: does the number of `Write` calls matter, or the number of bytes? Second: what should happen to the 3,000-byte body, which is bigger than A but smaller than B?

Step 3: what you should see

Request	Write calls	SOCKET lines (bytes, in order)	Header says
`/pieces?n=100`	100	**1** line: 1,119	`Content-Length: 1000`
`/pieces?n=300`	300	**1** line: 3,144	chunked
`/pieces?n=1000`	1,000	**3** lines: 4,096 then 4,096 then 1,973	chunked

Renee Banner & Alex Guerra

Request	Write calls	SOCKET lines (bytes, in order)	Header says
`/big?kb=1`	1	**1** line: 1,143	`Content-Length: 1024`
`/big?kb=64`	1	**3** lines: 4,096 then 61,572 then 7	chunked

In the `pieces` rows you made 100, then 300, then 1,000 `Write` calls and the kernel received 1, 1 and 3 `write()` calls. In the `big` rows you made **one** call each time and the kernel received 1 and 3.

Step 6: explain it in your own words

1. You called `Write` 1,000 times in `pieces?n=1000`, but the kernel received 3 calls. Where were the bytes in between? Name the boxes.
    a. Each 10-byte Write was copied into Buffer A (2,048 bytes, owned by the request).
    b. When A filled, it emptied through the chunkWriter, which built the header once and added chunk framing, into Buffer B (4,096 bytes, owned by the connection).
    c. B collected data until it was full, and only then did Go call write() to move bytes into the kernel send buffer.
2. In `pieces?n=300`, the header was built during the handler, but the `SOCKET` line came after `handler returns`. Explain why with the sizes of A and B.
    a. The body is 3,000 bytes. That is bigger than A (2,048), so A overflowed during the handler. The chunkWriter built the header at that point (moment 2) and chose chunked, because the handler had not finished and the length was unknown. The overflowed bytes moved into B. But B holds 4,096 bytes, and the total (about 3,000 body bytes plus header and framing, roughly 3,144) is smaller than that, so B never filled. With neither buffer forcing a send and no Flush(), nothing reached the kernel until the request finished.
3. A friend says: "I should join my small strings into one big string before writing, to avoid many system calls." Based on this experiment, is the advice necessary? When would it still be useful?
    a. to avoid system calls. Your code decides how many bytes are written, while the buffers decide how many write() calls happen. In the experiment, 1,000 small writes caused only 3 system calls, so the buffers already batch

Experiment 4: Flush, the only way to empty the buffers early

Goal: see that a handler can finish its writing and still keep the client waiting, and that `Flush()` is the tool that changes this.

The handler under test is `hold`. It writes 100 bytes, then sleeps (the default is 3 seconds, and you will use 1.5 seconds so you do not wait long). With `flush=1`, it calls `Flush()` right after writing and before sleeping. Think of the sleep as slow work: a database query, a call to another service, anything that takes time.

Renee Banner & Alex Guerra

Step 1: predict

Two runs: `/hold?flush=0&pause=1500` and `/hold?flush=1&pause=1500`. For each, write down:

1. How long will the client wait before it receives its first byte: about 0, or about 1.5 seconds?
    a. flush=0: about 1.5 s. flush=1: about 0 s.
2. In the log, will a `SOCKET` line appear **before** or **after** the line `sleeping 1.5s`?.
    a. the SOCKET line comes after the sleep.
3. `Content-Length` or `chunked`? (Remember what the chunkWriter knows at the moment it builds the header.)

    a. Content-Length

Step 6: explain it in your own words

1. What does `Flush()` empty: A, B, or both? Use the log (the 47-microsecond gap and the size of the `SOCKET` line) as evidence.
    a. Flush() empties both. The SOCKET line appeared 47 µs after "Flush() requested", and it was 231 bytes: header plus chunk framing plus the 100 body bytes. So the bytes left A (header built) and B (written to the kernel) in one call.
2. Why did adding `Flush()` change the header from `Content-Length` to `chunked`? Which of the three header moments is the one that decides it?
    a. At the flush the handler hadn't returned, so the length was unknown and the chunkWriter chose chunked. Moment 2 (header bytes built) is the one that decides it.
3. A developer says: "My handler returns quickly, so there is no need to flush." Under what condition is the developer right? (Hint: which log line told you when data left in `flush=0`?)

    a. The developer is right when the response is small enough to fit in A and nothing slow happens between the Write and the return. In flush=0, the log shows that data left only at the moment handler returns, so a quick return means an immediate send anyway.

Experiment 5: the header you cannot take back

Goal: prove with your own eyes that headers and the status are fixed at header moment 1, and learn the pattern that lets a handler report an error properly.

This experiment has three parts. Part A is about response headers. Part B is the famous problem: an error found after the response has started. Part C is the fix.

Predict: will the client see `X-Before`? Will it see `X-After`? Use the header moments to decide.

the client sees X-Before but not X-After.

Predict (write all of it down):

1. What status code will `curl` show: 200 or 500?
    a. 200.
2. What will the body contain: just the error text, or the half page plus the error text?
    a. The half page plus the error text.

Renee Banner & Alex Guerra

3. Will `http.Error` set `Content-Type: text/plain` and `X-Content-Type-Options: nosniff`, as it normally does?
    a. No. It will be text/html
4. Will the log show any warning from Go?

    a. Yes, in the server log

Part C: the safe pattern

The fix is simple: **do not touch `w` until you know the response will succeed.** Build the response in your own memory first. The handler `safeError` does this:

Predict: what will the status, the `Content-Type` and the body be this time?

text/plain and internal server error

Part D: apply it to a real handler

Here is a typical handler. It has the problem from Part B:

```
func reportPage(w http.ResponseWriter, r *http.Request) {
    tmpl, err := template.ParseFiles("report.html")
    if err != nil {
        http.Error(w, "Internal Server Error", http.StatusInternalServerError)
        return
    }
    if err := tmpl.Execute(w, data); err != nil {
        http.Error(w, "Internal Server Error", http.StatusInternalServerError)
    }
}
```

The `ParseFiles` error is handled correctly because nothing was written yet. The `Execute` error is not: `Execute` writes the page into `w` as it goes, so if it fails halfway, the status is already 200.

Your task: rewrite the second half so that a failure in `Execute` produces a proper 500 response. Use `bytes.Buffer`. Write your solution on paper or in a file, then compare it with the model answer in the instructor notes at the end. Do not look first.

```
func reportPage(w http.ResponseWriter, r *http.Request) {
        tmpl, err := template.ParseFiles("report.html")
        if err != nil {
                log.Printf("parse: %v", err)
                http.Error(w, "Internal Server Error", http.StatusInternalServerError)
                return
        }
```

Renee Banner & Alex Guerra

```
        var buf bytes.Buffer
        if err := tmpl.Execute(&buf, data); err != nil {
                log.Printf("execute: %v", err)
                http.Error(w, "Internal Server Error", http.StatusInternalServerError)
                return
        }

        w.Header().Set("Content-Type", "text/html; charset=utf-8")
        buf.WriteTo(w)
}
```

Part E: explain it in your own words

1. In Part A, the client had received nothing yet when `X-After` was set. Why was it already too late?
    a. The header map is copied at moment 1 (the first Write), not when bytes leave. “Too late” is measured in header moments, not in bytes sent.
2. In Part B, name the header moment at which the status became 200. Which log line shows it?
    a. Moment 1 is the line first Write: net/http calls WriteHeader(200) for you; status fixed here, right after the handler started.
3. Why does `safeError` get the right status when `lateError` does not? Answer in terms of "before moment 1" and "after moment 1".
    a. In safeError, http.Error is the first thing to touch w, so its status and header changes happen before moment 1 and take effect. In lateError, they happen after moment 1 and are ignored.
4. The safe pattern holds the whole page in memory before sending. For a 500 MB file download, why would that be a bad idea? What kind of check could you do before the first `Write` instead? (There is no single right answer, but think about what you can know in advance.)
    a. A 500 MB file would use 500 MB of memory per request, which is a scaling and denial-of-service risk. Instead, check beforehand what you can know cheaply: does the file exist, can it be opened, what is its size

Experiment 6 (optional): proof from the operating system

Part B: watch the kernel buffer fill

Renee Banner & Alex Guerra

Step 3: predict. The handler `big` makes one `Write` call of 4,194,304 bytes. While the client refuses to read:

1. Will a `SOCKET` line for the 4 MB body appear in the log, or not yet?
    a. Not yet
2. Will the line `handler returns` appear?
    a. No, the handler is blocked inside w.Write.
3. Where will the unsent bytes be sitting?

    a. Mostly in the kernel send buffer

Step 4: run it. Open a third terminal (Terminal 3) and start the client:

Explain it in your own words

1. In Part A, what does the number after `=` mean? Why does `= 143` not prove that the client received 143 bytes?
    a. The write() can't finish until the kernel has accepted all the bytes. The client wasn't reading, so the kernel's send buffer filled up and was the box that blocked the handler.
2. In Part B, why did the handler's `Write` call not return for 8 seconds? Name the box that was full.
    a. The kernel send buffer was full, so Write blocked until the connection died after 8 seconds.
3. How many kernel send buffers are there: one per request, or one per connection? What does your answer imply for HTTP/1.1 keep-alive connections that handle many requests one after another?

    a. One per connection (one per socket). With keep-alive, the same send buffer is reused for every request, as is Buffer B. A slow client can therefore still be draining the previous response when the next one is written, which delays it. This is why servers set WriteTimeout.

Final exercises

These three exercises combine everything. Do them in order and do not skip the predictions.

Exercise A: simulate the buffers on paper

The request is `/drip?size=700&count=5&pause=300`: five writes of 700 bytes. Draw this table on paper and fill in how many bytes you think are in each place after each event. Ignore the exact size of the header and framing: use "about" numbers. Recall: A holds 2,048 bytes and B holds 4,096 bytes.

After this event	Bytes in A	Bytes in B	Sent to the kernel
`Write #1` (700 bytes)	700	0	0
`Write #2`	1400	0	0

Renee Banner & Alex Guerra

After this event	Bytes in A	Bytes in B	Sent to the kernel
`Write #3` (this one crosses 2,048)	52	2048	0
`Write #4`	752	2048	0
`Write #5`	1452	2048	0
The handler returns	0	3500 + header bits	3644

Then answer these before running anything:

1. How many `SOCKET` lines will appear, and **when** (during the handler, or after it returns)?
    a. One Socket line
2. Will the header say `Content-Length` or `chunked`?

    a. Chunked

Now run it and check your answers against the log:

```
curl -s -o /dev/null 'localhost:8080/drip?size=700&count=5&pause=300'
curl -s -D - -o /dev/null 'localhost:8080/drip?size=700&count=5&pause=300' | grep -iE 'content-length|transfer-encoding'
```

The body is 3,500 bytes. If your table is right, the total of what reaches the socket should be the 3,500 body bytes plus roughly 140 bytes of header and framing. If your predictions were wrong, find the exact row of your table where your picture and the log disagree. That row is what you misunderstood.

Exercise B: predict a longer one

The request is `/drip?size=1500&count=6&pause=300`: six writes of 1,500 bytes, 9,000 bytes in total.

Predict, then run it with `curl -s -o /dev/null 'localhost:8080/drip?size=1500&count=6&pause=300'`:

1. At which `Write` number does the first `SOCKET` line appear? (Hint: the log shows no number for `Write #5` or `#6`, but each `Write` happens 0.3 seconds after the previous one, so you can work out the number from the clock. `Write #1` is at 0 seconds.)
    a. at Write #3,
2. Will there be a second `SOCKET` line **during** the handler? At which `Write`?
    a. at Write #6,
3. How many `SOCKET` lines in total, and what is the approximate size of the last one?

    a. Three SOCKET in total. The last one is about 973 bytes

Exercise C: draw it from memory

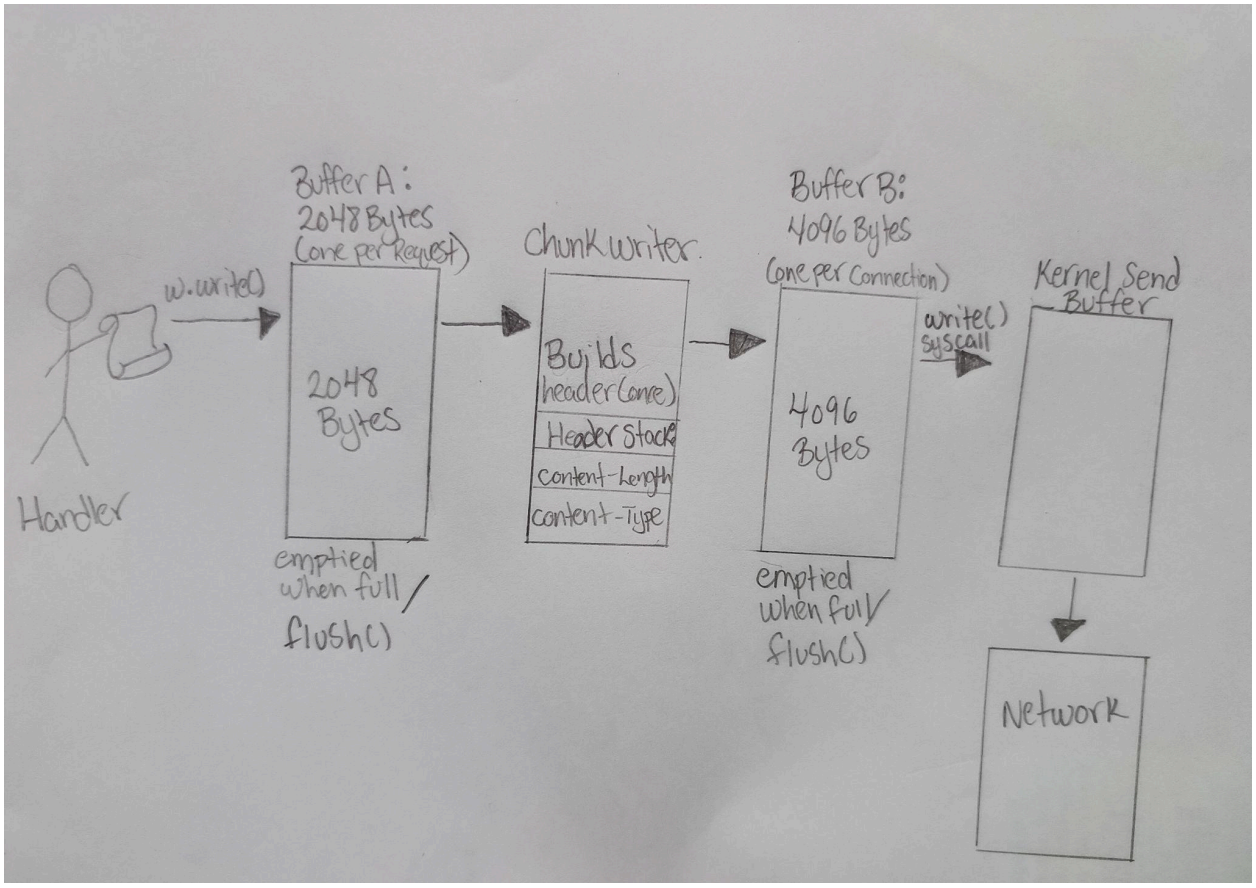
Close this document. On a blank page, draw the response path from your handler to the network **without looking at the diagram**. Your drawing must show:

- every box in the path, with its name, size and owner (one per request, or one per connection, or the kernel),

Renee Banner & Alex Guerra

- an arrow between each pair of boxes,
- for each buffer, the **three** things that make it empty,
- the three header moments, marked at the places where they happen,
- the two places where the lab's instruments (`HANDLER` and `SOCKET` lines) are attached.

Then open the model section and compare. Fix your drawing in a different colour. Keep it: a drawing you made yourself is the best revision sheet for this topic.



Reflection questions

Answer in your own words, in complete sentences. Two or three sentences each is enough. You may use your drawings.

1. How many buffers hold your response bytes between `w.Write()` and the network? List them in order, with sizes and owners.
    a. Three buffers. Buffer A is 2,048 bytes and owned by the request. Buffer B is 4,096 bytes and owned by the connection. The kernel send buffer is OS-sized and also per connection. The chunkWriter sits between A and B but holds nothing.
2. What are the three events that empty Buffer A? What are the three that empty Buffer B? (They look similar. Say how they differ.)
    a. A empties when it is full, on Flush(), or when the handler returns. B empties when it is full, on Flush(), or when the request is finished. The difference is that A's emptying

moves bytes inside Go and builds the header (moment 2). B's emptying is the only one that makes bytes leave for the kernel (moment 3).

3. Explain the difference between header moments 1, 2 and 3, using the log lines from Experiment 1 and Experiment 3 (`/pieces?n=300`) as evidence.
    a. In Experiment 1, moment 1 is the "first Write" line, moment 3 is the SOCKET line, and moment 2 is invisible between them. In /pieces?n=300, A overflowed during the handler (header built, chunked chosen), but B wasn't full, so the SOCKET line came only after the handler returned. This shows that moments 2 and 3 are separate events.
4. A handler writes 100 bytes and then does a 5-second database query. What does the client see during those 5 seconds, and what single line of code would change that? What is the cost of that line?
    a. The client sees nothing for 5 seconds, because the 100 bytes sit in Buffer A. Adding http.NewResponseController(w).Flush() after the Write sends them immediately. The cost is that the response becomes chunked instead of Content-Length and the server makes an extra system call.
5. Why can a handler not report a failure halfway through writing a response? Describe the safe pattern in your own words.
    a. Writing to w fixes the status and headers at moment 1, so a later http.Error is ignored. The safe pattern is to build the response in a bytes.Buffer, check for errors, and only then write to w. On failure, http.Error is the first thing to touch w.
6. In Experiment 6 (or from your reading of its explanation), why can a slow client keep your handler from returning?
    a. If the client doesn't read, the kernel send buffer fills and write() blocks. The handler's Write can't return, so the handler can't return, which makes write timeouts important.
7. Which of the six experiments changed your mental model the most, and what did you believe before it?
    a. the "client had received nothing even though my handler finished" idea changed my model the most. I believed Write meant "sent", and Experiment 1 showed it only means "copied into Buffer A".
8. Write one question you still have about the response path. (This is not graded for correctness.)
    a. How does http.ServeContent or io.Copy with a file avoid holding the whole file in memory, and how does it interact with Buffer A and B?

Submission checklist

- ☐ Your predictions for Experiments 1 to 5, written **before** you ran each one, next to the real results
- ☐ The filled table from Exercise A and your answers to Exercise B
- ☐ Your hand-drawn diagram from Exercise C
- ☐ Your answers to the explain-in-your-own-words questions of each experiment
- ☐ Your answers to the reflection questions
- ☐ Your rewritten `reportPage` handler from Experiment 5, Part D
- ☐ (Optional) The `strace` lines from Experiment 6, Part A